

Can Online Browsing Behaviour Predict Purchase Intention?

Stage 1: Logistic Regression Classification

CS-C3240 Machine Learning D

1 Problem Formulation

The aim of this project is to determine whether behavioural and contextual information recorded during an online shopping session can be used to predict whether the session results in a purchase. The application is formulated as a supervised binary classification problem.

Each data point represents one online shopping session. For session i , the input is a multidimensional feature vector $\mathbf{x}_i$, and the corresponding label is

$$y_i = \begin{cases} 1, & \text{if the session resulted in a purchase,} \\ 0, & \text{otherwise.} \end{cases}$$

The objective is to learn a classifier

$$f(\mathbf{x}) \rightarrow \{0, 1\}$$

that generalises to previously unseen shopping sessions. In practical terms, the model attempts to distinguish potential buyers from visitors who leave the website without purchasing.

1.1 Dataset

The project uses the *Online Shoppers Purchasing Intention Dataset* from the UCI Machine Learning Repository. The original dataset contains 12,330 observations, 17 predictor variables, and the binary target **Revenue**.

The predictors describe both browsing behaviour and session context. Behavioural variables include the numbers and durations of administrative, informational, and product-related page visits, together with bounce rate, exit rate, page value, and proximity to a special day. Contextual variables describe month, operating system, browser, region, traffic type, visitor type, and weekend status. A complete feature description is provided in Appendix A.

The original target distribution is strongly imbalanced: 1,908 sessions (15.5%) resulted in a purchase and 10,422 (84.5%) did not. This is important for model development because a classifier trained directly on the original distribution may favour the majority non-purchase class.

2 Methods

2.1 Data Quality and Preprocessing

The raw data were first inspected for missing values and exact duplicate rows. No missing values were found, so no imputation was required. A total of 125 exact duplicate rows were identified and removed, reducing the modelling dataset from 12,330 to 12,205 observations. After duplicate removal, the dataset contained 10,297 non-purchase sessions (84.37%) and 1,908 purchase sessions (15.63%).

All 17 predictor variables were retained for the Stage 1 model, so no feature elimination was performed. This provides a baseline using the complete set of behavioural and contextual variables available in the dataset. The features were instead separated into numerical and categorical groups so that preprocessing could be applied according to the meaning of each variable.

The predictors were separated according to their meaning rather than their raw storage type. Ten variables representing counts, durations, rates, page value, and special-day proximity were treated as numerical. Seven variables, **Month**, **OperatingSystems**, **Browser**, **Region**, **TrafficType**, **VisitorType**, and **Weekend**, were treated as categorical. In particular, variables such as browser and region are represented by numbers in the raw dataset, but these numbers are category identifiers rather than continuous quantities.

Numerical predictors were standardised using **StandardScaler**. For a numerical feature x , this transformation is

$$z = \frac{x - \mu}{\sigma},$$

where μ and σ are estimated from the training data. This places numerical predictors measured on different scales onto comparable scales.

Categorical predictors were transformed using one-hot encoding rather than being treated as continuous numerical values. Unknown categories were configured to be ignored so that a category occurring in validation or test data

but not in the training subset does not cause an error.

The standardisation and one-hot encoding operations were implemented together with the classifier using a Scikit-learn pipeline. Consequently, preprocessing parameters are estimated from the training data only and are subsequently applied to validation and test observations. This avoids information from the evaluation sets influencing model fitting.

2.2 Training-Class Balancing

Since only 15.63% of the cleaned observations correspond to purchases, the training data were balanced using random undersampling of the majority class. Importantly, balancing was performed after the training, validation, and test split and was applied only to the training set.

Before balancing, the training subset contained 7,207 non-purchase sessions and 1,336 purchase sessions. All 1,336 purchase observations were retained, and 1,336 observations were randomly sampled without replacement from the non-purchase class. The resulting training set therefore contains 2,672 observations, equally divided between the two classes.

Validation and test data were not balanced. They retain the natural purchase rate of approximately 15.6%, allowing model performance to be assessed on data that preserve the original class distribution. Balancing only the training set also prevents information from the validation or test sets from influencing the sampling procedure.

Table 1: Data partitioning and class distribution.

Subset	Sessions	No Purchase	Purchase
Cleaned dataset	12,205	10,297	1,908
Training, before balance	8,543	7,207	1,336
Training, after balance	2,672	1,336	1,336
Validation	1,831	1,545	286
Test	1,831	1,545	286

2.3 Training, Validation and Test Design

After duplicate removal, the 12,205 observations were divided using a stratified 70/15/15 split. This produced 8,543 training observations, 1,831 validation observations, and 1,831 test observations.

Stratification was used to preserve approximately the same purchase rate in each subset before training-set balancing: 15.64% in the training set and 15.62% in both the validation and test sets. The 70% training allocation provides the largest portion of the available observations for learning the model, while the two 15% held-out subsets provide separate data for model validation and final testing.

The validation set is used to assess the model during development without using the test set to make modelling decisions. The test set is reserved for final evaluation. A fixed random state of 42 was used to make the split and random undersampling reproducible.

2.4 Logistic Regression and Hypothesis Space

Logistic Regression was selected as the Stage 1 method because the target is binary and the method provides a relatively simple and interpretable baseline. It belongs to the class of linear predictors: the model learns a linear function of the transformed input features and maps it to a probability using the sigmoid function.

For feature vector $\mathbf{x}$, the probability of a purchase is modelled as

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}},$$

where $\mathbf{w}$ contains the learned coefficients and b is the intercept. A decision threshold of 0.5 is used to convert the predicted probability into a class prediction.

Logistic Regression is suitable for Stage 1 because it directly models binary class probabilities while keeping the hypothesis space less complex than nonlinear alternatives. It therefore provides a useful reference method for later comparison with more flexible classifiers.

2.5 Loss Function

The model is fitted by minimising logistic loss, also known as binary cross-entropy. For N training observations,

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)],$$

where $p_i = P(y_i = 1 \mid \mathbf{x}_i)$ is the predicted probability that session i results in a purchase.

Logistic loss is appropriate for this problem because the model predicts a probability for a binary outcome. The loss penalises incorrect probability estimates, with confident incorrect predictions receiving a larger penalty.

2.6 Model Evaluation Procedure

Although the Stage 1 model is trained on a balanced sample, validation and testing are performed on the original imbalanced distribution. Therefore, classification error is considered together with precision, recall, F1-score, ROC-AUC, and the confusion matrix.

Accuracy alone is potentially misleading in this application because a large majority of sessions do not result in purchases. Precision measures the proportion of predicted purchasers that actually purchase, while recall measures the proportion of actual purchasers identified by the model. F1-score summarises the trade-off between precision and recall, while ROC-AUC evaluates ranking performance across classification thresholds.

The same fitted preprocessing pipeline and Logistic Regression model are applied to the validation and test sets without refitting the preprocessing transformations on either set. The complete implementation and generated evaluation outputs are included in [Appendix C](#).

3 Use of AI

Generative AI tools were used as supporting tools during the project. ChatGPT GPT-5.6 Sol was used during project planning, to discuss preprocessing, to assist with the structure and review of Python code, and to assist with the drafting of the Stage 1 report.

A Feature Description

Table 2: Predictor variables used by the Stage 1 model.

Feature	Description
Administrative	Number of administrative pages visited.
Administrative_Duration	Time spent on administrative pages.
Informational	Number of informational pages visited.
Informational_Duration	Time spent on informational pages.
ProductRelated	Number of product-related pages visited.
ProductRelated_Duration	Time spent on product-related pages.
BounceRates	Average bounce rate of visited pages.
ExitRates	Average exit rate of visited pages.
PageValues	Average page value associated with visited pages.
SpecialDay	Closeness of the session to a special day.
Month	Month in which the session occurred.
OperatingSystems	Operating-system category.
Browser	Browser category.
Region	Geographical region category.
TrafficType	Traffic-source category.
VisitorType	Visitor category.
Weekend	Whether the session occurred during a weekend.

B Evaluation Outputs

The following outputs were generated by the Stage 1 implementation. They are included here for reproducibility and later comparison.

Table 3: Performance produced by the Stage 1 implementation.

Subset	Accuracy	Precision	Recall	F1	ROC-AUC
Balanced training	0.8241	0.8620	0.7717	0.8144	0.9143
Validation	0.8443	0.5011	0.7622	0.6047	0.9035
Test	0.8520	0.5180	0.7552	0.6145	0.8954

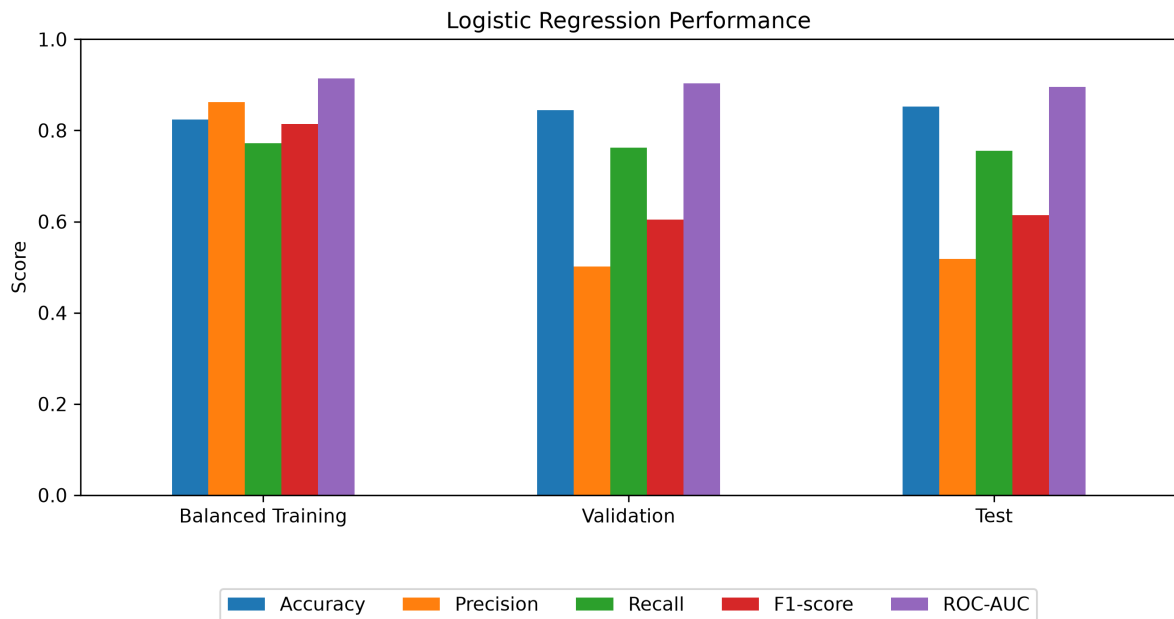


Figure 1: Logistic Regression performance across the balanced training, validation, and test subsets.

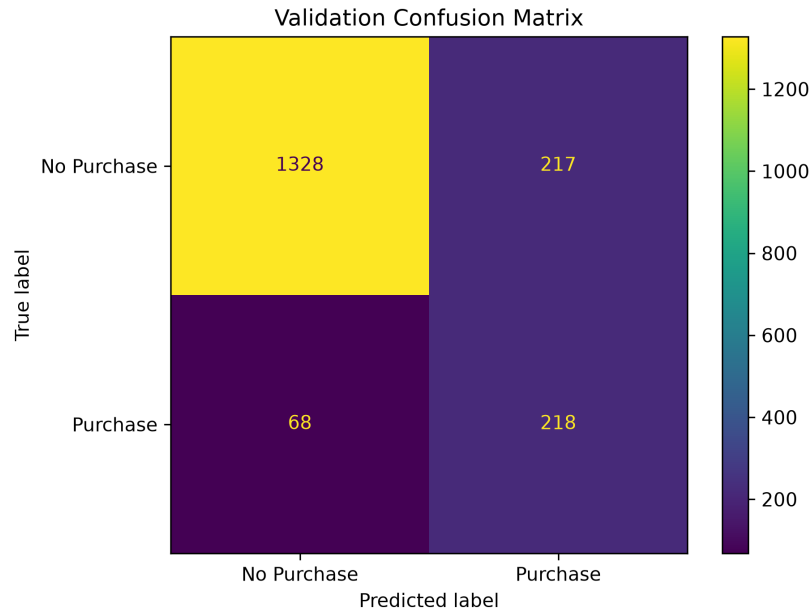


Figure 2: Confusion matrix on the validation set. The model correctly identified 218 of 286 purchasing sessions.

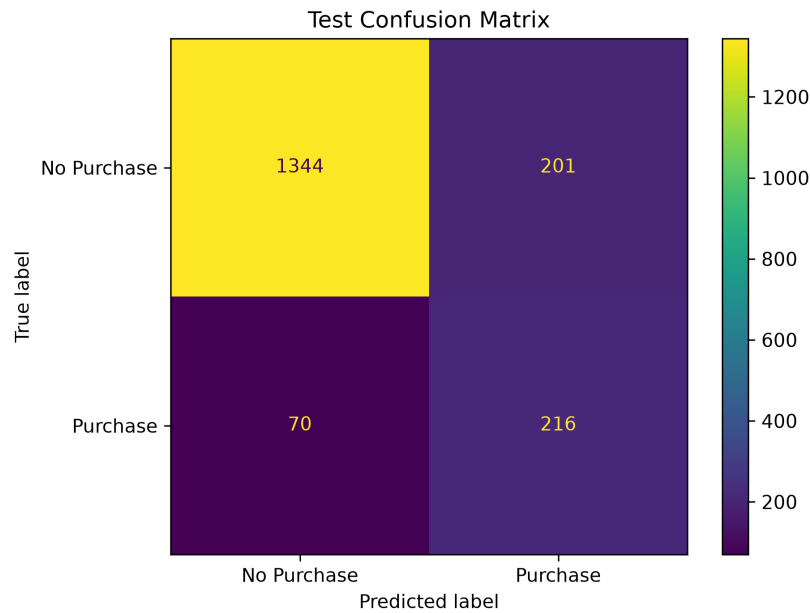


Figure 3: Confusion matrix on the test set. The model correctly identified 216 of 286 purchasing sessions.

C Source Code

The complete Python implementation used to preprocess the data, construct the training, validation, and test sets, balance the training data, train Logistic Regression, and evaluate the model is included below.

```

1 from pathlib import Path
2
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.compose import ColumnTransformer
8 from sklearn.pipeline import Pipeline
9 from sklearn.preprocessing import OneHotEncoder, StandardScaler
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.metrics import (
12     accuracy_score,
13     precision_score,
14     recall_score,
15     f1_score,
16     roc_auc_score,
17     confusion_matrix,
18     ConfusionMatrixDisplay,

```

```

19 )
20
21
22 # -----
23 # Load and clean data
24 # -----
25
26 PROJECT_ROOT = Path(__file__).resolve().parents[2]
27 DATA_FILE = PROJECT_ROOT / "data" / "online_shoppers_intention.csv"
28 FIGURE_DIR = PROJECT_ROOT / "figures"
29 FIGURE_DIR.mkdir(parents=True, exist_ok=True)
30
31 df = pd.read_csv(DATA_FILE)
32
33 if df.isna().sum().sum() > 0:
34     raise ValueError("Dataset contains missing values.")
35
36 df = df.drop_duplicates().reset_index(drop=True)
37
38
39 # -----
40 # Features and target
41 # -----
42
43 numerical_features = [
44     "Administrative",
45     "Administrative_Duration",
46     "Informational",
47     "Informational_Duration",
48     "ProductRelated",
49     "ProductRelated_Duration",
50     "BounceRates",
51     "ExitRates",
52     "PageValues",
53     "SpecialDay",
54 ]
55
56 categorical_features = [
57     "Month",
58     "OperatingSystems",
59     "Browser",
60     "Region",
61     "TrafficType",
62     "VisitorType",
63     "Weekend",
64 ]
65
66 features = numerical_features + categorical_features
67
68 X = df[features].copy()
69 y = df["Revenue"].astype(int)
70
71 for column in categorical_features:
72     X[column] = X[column].astype(str)
73
74
75 # -----
76 # Stratified 70/15/15 train-validation-test split
77 # -----
78
79 X_train, X_temp, y_train, y_temp = train_test_split(
80     X,
81     y,
82     test_size=0.30,
83     random_state=42,
84     stratify=y,
85 )
86
87 X_val, X_test, y_val, y_test = train_test_split(
88     X_temp,
89     y_temp,
90     test_size=0.50,
91     random_state=42,
92     stratify=y_temp,
93 )
94
95
96 # -----
97 # Balance training set using random undersampling
98 # -----
99
100 train = X_train.copy()
101 train["Revenue"] = y_train
102
103 buyers = train[train["Revenue"] == 1]
104 non_buyers = train[train["Revenue"] == 0].sample(
105     n=len(buyers),
106     random_state=42,
107 )
108
109 train_balanced = pd.concat([buyers, non_buyers]).sample(
110     frac=1,
111     random_state=42,
112 ).reset_index(drop=True)

```

```

113 X_train_balanced = train_balanced.drop(columns="Revenue")
114 y_train_balanced = train_balanced["Revenue"]
115
116
117
118 # -----
119 # Preprocessing and Logistic Regression
120 # -----
121
122 preprocessor = ColumnTransformer(
123     [
124         (
125             "numerical",
126             StandardScaler(),
127             numerical_features,
128         ),
129         (
130             "categorical",
131             OneHotEncoder(handle_unknown="ignore"),
132             categorical_features,
133         ),
134     ]
135 )
136
137 model = Pipeline(
138     [
139         ("preprocessor", preprocessor),
140         (
141             "classifier",
142             LogisticRegression(
143                 max_iter=2000,
144                 random_state=42,
145             ),
146         ),
147     ]
148 )
149
150 model.fit(X_train_balanced, y_train_balanced)
151
152
153 # -----
154 # Evaluation
155 # -----
156
157 def evaluate(model, X, y, name):
158     predictions = model.predict(X)
159     probabilities = model.predict_proba(X)[:, 1]
160
161     return {
162         "Dataset": name,
163         "Accuracy": accuracy_score(y, predictions),
164         "Error Rate": 1 - accuracy_score(y, predictions),
165         "Precision": precision_score(y, predictions),
166         "Recall": recall_score(y, predictions),
167         "F1-score": f1_score(y, predictions),
168         "ROC-AUC": roc_auc_score(y, probabilities),
169     }
170
171
172 results = pd.DataFrame(
173     [
174         evaluate(
175             model,
176             X_train_balanced,
177             y_train_balanced,
178             "Training",
179         ),
180         evaluate(
181             model,
182             X_val,
183             y_val,
184             "Validation",
185         ),
186         evaluate(
187             model,
188             X_test,
189             y_test,
190             "Test",
191         ),
192     ]
193 )
194
195 print(results.round(4).to_string(index=False))
196
197
198 # -----
199 # Confusion matrices
200 # -----
201
202 def save_confusion_matrix(model, X, y, name):
203     predictions = model.predict(X)
204     cm = confusion_matrix(y, predictions)
205
206     display = ConfusionMatrixDisplay(

```

```

207     confusion_matrix=cm,
208     display_labels=["No Purchase", "Purchase"],
209 )
210
211 display.plot(values_format="d")
212 plt.title(f"{name} Confusion Matrix")
213 plt.tight_layout()
214
215 plt.savefig(
216     FIGURE_DIR / f"{name.lower()}_confusion_matrix.png",
217     dpi=300,
218     bbox_inches="tight",
219 )
220
221 plt.close()
222
223
224 save_confusion_matrix(
225     model,
226     X_val,
227     y_val,
228     "Validation",
229 )
230
231 save_confusion_matrix(
232     model,
233     X_test,
234     y_test,
235     "Test",
236 )

```